The Legend of PPO: Learning to Fight in Hyrule

Jordy Kieto

Code, claim audit and provenance: https://github.com/JordyKieto/ppo-hyrule

Models: https://huggingface.co/jordykieto/ppo-hyrule

Video: https://youtu.be/FBD1wEsdY3E



Figure 1: One episode of the policy defeating Queen Gohma, from the full-health start, at the frames the mechanism turns on. She takes damage only while stunned, and a stun has to be manufactured: the agent shoots her out of the ceiling with a slingshot seed, then hits her on the floor, and later uses Deku Nuts to the same end. The last panel is the end of the recording: the episode terminates on her defeat, so the capture stops while she is going over and before she disintegrates. All six panels are identically gamma-corrected to lift the shadows of the first two, with no other adjustment. The full unedited episode is linked in §11, and every attempt made while recording, won or lost, is listed beside it.

Model-free reinforcement learning from live game memory in *The Legend of Zelda: Ocarina of Time*, and four things that did not beat it.

Technical report, 1 November 2026. Every quantitative statement here carries an evidence class in the claim audit and traces to an artefact in the provenance record. Nothing below should be read as a result unless it is marked **M**, measured, there. Claims this project made internally and later overturned are listed in §12 rather than deleted, which is a deliberate part of the report rather than an oversight.

Abstract

We train agents to fight in an unmodified commercial 3D game, *Ocarina of Time*, running under the Dolphin emulator. The agent never sees a pixel. It reads the emulator's memory directly, giving a structured view of the world: every actor's position, velocity, health and decoded animation routine. **This removes the pixel-processing problem, not the perception problem.** The agent still has to work out, from a variable-length vector, which actor is the boss, what her decoded action routine implies about what she is about to do, and what the geometry means in a frame that rotates with the camera. What it does not have to do is recover any of that from an image.

Two tasks. The first, a three-enemy melee encounter, is solved by recurrent PPO in under five hours on twenty CPU machines. The second, the dungeon boss Queen Gohma, is not solved, and the useful content of this report is why. Gohma takes damage only while stunned, and a stun must be manufactured. From a full-health start a kill is ten sword hits spread across roughly five such openings, inside a fight

that takes a human 154 seconds. Our best policy defeats her on 8.33% of 456 evaluation episodes and removes 3.53 health on average, pooled over starts that begin at 5 to 10 of her 10 health.

Four alternatives were measured against that policy under matched conditions and **all four lost**: cloning human play, folding those demonstrations into the policy gradient, learning offline from return-conditioned trajectories, and planning inside a learned model. The masked-prediction result in §8 is not one of the four; it is a separate positive finding about representation, not a competitor to the policy. Behaviour cloning on 100 recorded human victories reaches 74.5% held-out next-action accuracy, nearly four times the majority-action floor, and takes 0.33 of her 10 health with 1 win in 80 attempts. Fifteen separate mechanisms for injecting those demonstrations into the policy gradient produced no arm trending toward the policy trained without them. Offline return-conditioned behaviour cloning reaches 1.31 health and 3.95% defeats at matched episode caps. A learned world model trained for 120 epochs ends further from "nothing changes" than it started and ignores the action entirely. Separately, a closed planning loop in which each of six machines trained its own smaller world model on its own freshly collected games produced dream-trained policies that won 0 of 607 real episodes.

The second half of the report is about measurement, because in this project the instrument was a larger source of error than any method. A defeat is a 1 to 10% binary event. Read through it, six consecutive versions of the boss policy appeared flat and two experiments were launched to break the apparent plateau. Read through the graded quantity the reward actually pays for, the same six versions rose **8.7 times**, every step statistically solid, and the two changes launched to break the apparent plateau were both measurable regressions. We give the per-start profile rather than a pooled rate, because the pooled rate moves with a curriculum we choose, and we record eleven claims this project published and then withdrew.

1 Contributions

- **A boss fight as an RL benchmark with the mechanism fully instrumented.** Not "sparse reward" in the abstract: a measured causal chain, mined from 100 human victories, in which 678 of 678 damage events required a stun, a stun converts from a projectile hit on only 40.6% of attempts in the state where the agent is most likely to attempt it, and 0% of 71 attempts in two other states (§4).
- **A measured profile of start-state dependence** rather than a pooled success rate. Defeat rate across six starts of the same task ranges from 28.95% to 0 of 76 episodes, ordered by how much of the fight the start hands the policy (§5).
- **Imitation measured against the policy that never saw a human.** Frame-level accuracy and closed-loop competence are not merely weakly related here, they are **anticorrelated across corpora** at r = -0.875 (§6). Two policies that both beat the boss, one human and one learned, agree on **3% of frames**.
- **A negative result on learned world models, carried to the point where the mechanism is named.** The model fails the action-shuffle control outright, so it is not a controllable dynamics model in any sense, which explains the dream-trained policies without appealing to the policy (§7).
- **One positive representation result, stated at the resolution it earns.** Masked prediction over the structured state improves **9 of 10 frozen downstream probes** against both an untrained network of identical architecture and a free-signal control, while the **pretraining objective itself remains worse than a trivial baseline**, on **one seed** (§8). Those are two different conclusions and we keep them apart.
- **A metric methodology, derived from its own failures.** Mix-invariance, selector resolution, the maximum-of-N bias, pooling as a measurement with its own failure modes, and the difference between a discount and a policy-gradient weight (§4.1), each with the specific wrong decision it caused.

1.1 What we do not claim

Stated here rather than buried, because the paper's own argument is that claims outrun evidence easily.

- **We do not claim generalisation to unseen start states.** The four starts excluded from training are all at half health and all beyond 368 units, so "held out" is collinear with "far" and no matched comparison exists.
- **We do not claim the credit horizon is the binding constraint.** We show it supplies almost no direct long-horizon credit. We did not run the experiment that would test whether fixing it helps, and the arms that raised the horizon moved other things at the same time.
- **We do not claim the approach-shaping term caused the plateau.** The dose-response is confounded by each version's starting level, and our own claim audit marks that reading confounded.
- **We do not claim the masked representation transfers to the boss.** The raw state it needs was never recorded for Gohma, so the experiment cannot currently be run.
- **We do not claim the entity-factored recurrent architecture is better than a flat network.** It was never ablated head to head. No architectural novelty is claimed anywhere in this report.
- **We do not claim the human's strategy is inferior.** The demonstrator won 100 of 100. What we show is that imitating those demonstrations, by fifteen distinct mechanisms of which nine ran long enough to fit a slope, did not improve a policy trained without them.
- **We do not claim a new algorithm.** Every method here is taken from the literature and applied.

2 Setup

2.1 Observation: live game memory

Reading an emulator's memory instead of its framebuffer is an established way to study console games [5, 23], and the closest precedent to what we do reads position, velocity and action state out of a fighting game's memory every frame [13].

The game is *The Legend of Zelda: Ocarina of Time* (GameCube build `NACE01`) under Dolphin 2603a with the video backend disabled. Observations are read zero-copy from Dolphin's shared memory arena at about 25 microseconds per step. Control runs at 10 Hz, one action every 6 vertical interrupts.

Every memory offset is taken from the OoT decompilation rather than from inspection, with the retail Player structure at the decompilation's offset minus `0x10`. One shared definition (`zelda_rl/state_layout.py`) is imported by both the data collector and every model, so the two cannot drift.

Reading structured state out of an emulator instead of rendering pixels is not our idea. Firoiu, Whitney and Tenenbaum (arXiv:1702.06230) did it for *Super Smash Bros. Melee* under the same emulator family, reading each player's position, velocity and action state from memory every frame; §14 states what we take from them and what we do not claim.

Reading memory is not a shortcut past the hard part, and it is worth being precise about which part it removes. It removes **rendering and pixel processing**. It does not remove perception. The observation is a variable set of actors over twenty fields each, 1,033 numbers for the boss task, and more than half of them describe slots that are empty at any given moment. Nothing in that vector is labelled: the policy has to learn which row is the boss, which rows are larvae worth ignoring, what the decoded action routine implies about what happens next, and what the distances and velocities mean in a frame that rotates with the camera. That is a perception problem, posed over structured state instead of over pixels.

What the structure buys is that the quantity mattering most for combat, an enemy's current **action routine**, is present at all. It is a function pointer we decode into a named behaviour state. A Deku Baba telegraphs its lunge; Gohma's eye turns red before she is vulnerable. Those are the cues a human plays on, and here they

are in the observation rather than three frames of animation away from it. That is the reason the environment is interesting rather than a reason it is easy.

Vocabulary, fixed once. A *machine* is one cloud pod and is also called a rank by the tooling. An *arm* is one configuration under test. An *update* is one iteration of the policy-gradient optimiser. *u* is a world unit, the game's own distance measure; Link is about 60 units tall and the boss room is roughly 1,000 units across.

2.2 Actions and reward

Actions are a discrete product of nine world-relative directions and a button set. The melee task uses 63 actions; the boss task needs 108, adding slingshot, Deku Nut, first-person aiming, and the Z-plus-shield and Z-plus-nut combinations. **Z-targeting** is the game's own lock-on, which fixes the camera and Link's facing on a chosen enemy; it is referred to by that name throughout because that is what the button does.

Reward is computed by one tracker (`zelda_rl/combat.py`) shared by the real environment and the learned one, so the two cannot disagree by construction.

The reward configuration is pinned per policy, not quoted from the current file. That file is edited between experiments, so a sentence describing "the reward" describes whatever it happens to be on the day it is read, which for a project whose central finding is that reward composition decided the outcome is not good enough. reward provenance resolves the exact coefficients for every reported policy from the commit it trained under, parsed out of version control rather than recalled. Below is the one that matters: **v15, which carries every headline number in this report.** It is the `simple` preset plus exactly one term.

Outcomes. These are what the task is about, and they are uncapped.

Table 1: The reward actually used to train every Gohma policy reported here, as a list of terms. Coefficients and caps are read from the training configuration of each run, not retyped; the per-version differences are in the version table of §5.3 and the full resolution is in the reward provenance document.

term	value	pays for
`g_boss_defeat`	**+60**	the kill
`g_boss_damage`	**+4.0**	per health point taken off her
`g_boss_hp_progress`	2.0	multiplier making her last health worth more than her first
`g_death_penalty`	30.0	dying. Scaled to **0** in v15 by a launch flag
`w_link_damage` / `w_link_heal`	0.25 / 0.05	per health unit lost or regained, so 4.00 per heart
`w_defeat` / `w_enemy_damage`	5.0 / 1.0	the generic combat outcomes, inherited

The stun chain. The one causal path to damage, and the largest shaping term in the reward.

Table 2: The experimental preset added after the reported results, for the five behaviours watched in play. It trained no policy in this report and is listed so it cannot be confused with the configuration above.

term	value	cap	note
`g_seed_stun`	**15.0**	60.0	a slingshot stun, **paid only when converted into damage**
`g_nut_stun`	2.0	10.0	a Deku Nut stun, same conversion gate
`g_stun_hit` / `w_stun_hit`	2.0 / 0.5		a sword hit landed while she is down
`g_nut_miss` / `g_seed_miss`	0.3 / 0.15		penalties, scaled up as that ammunition runs low

Positioning, defence and larvae. Everything else that pays at all.

Table 3: The melee task's reward, which is the configuration the solved task used.

term	value	cap
`g_approach`	**0.5**	3.0
`w_approach`, firing as approach-while-stunned	0.5	per-step clip ±0.5
`g_spacing`	0.02	1.5
`g_sword_ready`	0.02	1.0
`g_larva_kill` / `g_larva_kill_sling`	2.0 / 1.0	
`g_larva_damage` / `g_larva_damage_sling`	0.5 / 0.25	
`w_block` / `w_dodge` / `w_shield_face`	1.0 / 0.2 / 0.01	
`g_link_heal`	0.1	

Twenty terms are zeroed, and which ones is the interesting part. The whole slingshot-shaping cluster is off: `g_red_eye_seed`, `g_sling_red_eye`, `g_sling_ready`, `g_lock_red_eye`, `g_unfired_red_eye`, `g_drop_red_eye_shot`, `g_elevated_seed`. So are the pickups and grass (`g_seed_pickup`, `g_nut_pickup`, `g_grass_cut`), lock-on (`w_lockon`, `w_lock_step`) and the guard term. That cluster was removed on measurement, not on taste: §14 gives the version where it reached 88% of total reward magnitude while the policy died 64.5% of the time.

Three properties of this table are worth stating because they drive results later in the paper. **The stun credit is conditional**: `g_stun_on_convert` is on, so stunning her without landing a hit pays nothing, which is why the policy's stun counts and its damage move together. **`g_seed_stun` at 15.0 is by a wide margin the largest shaping term**, and §5.2 shows the policy earns it about once in twenty wins, which is the clearest evidence in this report that the red-eye shot is under-executed rather than under-rewarded. And **none of these terms is potential-based** in the sense of Ng, Harada and Russell, so none of them is guaranteed to leave the optimal policy unchanged; §9.1 returns to that.

A later experiment, not reported as a result in this paper, raises damage to 20 and rescales the shaping onto one scale. It is a separate preset precisely so the numbers above keep meaning what they meant.

2.3 Policy and optimisation

The policy is an entity-factored recurrent actor-critic: the actor table is pooled by a shared per-actor network, then a gated recurrent unit [8, 9] of width 256 feeds actor and critic heads. Training is PPO [30] with generalised advantage estimation [29], Adam [20] throughout, and truncated backpropagation through time over the recurrent core, following the practice established for recurrent PPO [25]. Across machines we average weights every update rather than averaging gradients. **We claim no architectural novelty and ran no head-to-head ablation**: this architecture was never compared against a flat network at scale, so it is a choice we made and not a finding, and nothing in the rest of the paper rests on it. That caveat is worth taking literally, because implementation choices in on-policy algorithms have been shown to account for differences as large as the ones usually attributed to the algorithm [2].

2.4 Compute, and what it cost

Reinforcement learning ran on fleets of 16-vCPU cloud machines, between 14 and 48 of them at a time and 109 at the largest, at $0.48 to $0.56 per machine-hour. World models ran on single H100 80GB cards at $3.49 to $3.99 an hour. **Nothing in this work used an A100**, although one pod was requested as one and

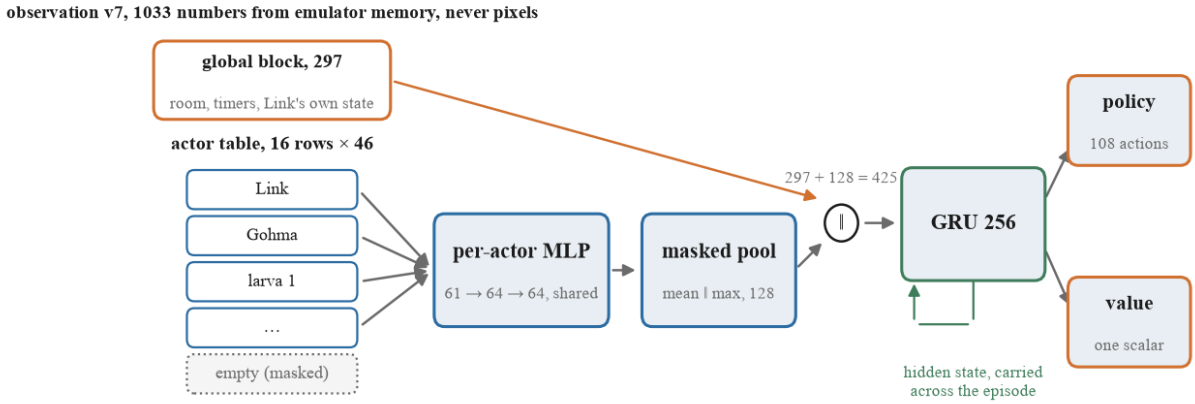


Figure 2: The policy. Each actor in the room becomes a row in a ragged table, every row goes through the same small network, and the rows are pooled with a mask so one set of weights handles any number of enemies. The pooled vector is concatenated with a global block that holds the room, the timers and Link's own state, and a recurrent core carries what has happened so far in the fight. Widths are read from the reported checkpoint, not from prose. We claim no architectural novelty and ran no head-to-head ablation.

the provider substituted an H100. Our own logs recorded this as "A100 or H100" for months, and that was wrong.

The whole project consumed about **2,300 machine-hours and $1,381**, split **$1,115 for the reinforcement learning track and $266 for the world model track**. It generated **509.6 million environment steps**, which is **14,154 hours of game time**, roughly 1.6 years of play, at about **$2.71 per million steps**. The melee task of §3 is the one phase costed from a verified artefact: **$51 and 4 hours 52 minutes** for the policy that solved it over 41.8 million steps, **$1.22 per million**, against $61.50 for the whole six-hour session including a discarded run. The rest of the project is twice as expensive per step, which is what a long tail of short diagnostic fleets costs rather than a change in efficiency. One reconciliation to state rather than hide: the step count rebuilt from the exported histories is 44.35 million, not the 41.8 million our logs record, and we have not resolved which is right.

The caveat matters more than the totals, and it is a finding about provenance rather than about cost. Only **18% of that spend is measured**. Our provider keeps a per-pod billing ledger that survives termination, and we read **43.7 H100-hours and $172.69** of world model training out of it, pod by pod, months later. That ledger covers GPU machines only: a CPU pod billing at the moment we queried it returns no records, and terminated pods are not listed by any endpoint. The remaining **82%** is reconstructed as machines times hourly rate times the span between the first and last timestamp our experiment tracker holds for that session. Rates and machine counts come from our own launch logs and are reliable; the durations are not, and they err low, since bootstrap and idle time falls outside that span. Two fleets are not reconstructible at all: a 109-machine data collection whose only surviving trace is the shard directories it wrote, and a 34-machine fleet the provider terminated without a reason while we were not watching. We report those as missing rather than as estimates. provenance record section 19 carries the per-phase table.

3 Task one: the melee encounter is solved

Three Deku Babas in the first room of the Deku Tree, with the room exit behind the agent so that walking out is an available failure. Success is all three defeated.

At global update 150 the policy clears **10 of 10 start states locally with 0 deaths**, six of them without taking damage (**Q: one watched episode per state, not a rate**). Over the same updates the aggregator

machine, the only one whose history was exported, reads **72.9% success and 80.7% of Babas killed (M)**. A 78% / 83% fleet mean over all twenty machines appears in our logs and **is not recoverable from the exported artefacts**, so we quote the recoverable number. Training ran 41.8 million environment steps, about 1,160 hours of game time.

The learned behaviour is recognisable as play rather than as exploitation: it walks in from the entrance, wakes Babas that are underground, locks on, and times attacks against the lunge animation. The clip accompanying this report (§11) is one such episode: the episode runs 26.7 seconds and the clip 29.7, the difference being the hold after the last kill. Three kills, at 7.3, 22.4 and 26.8 seconds.

This task matters mainly as calibration. It establishes that the observation, the action space and the optimiser are jointly sufficient to learn real combat in this engine, which is what makes the boss result a statement about the boss rather than about the setup.

4 Task two: what Queen Gohma actually requires

Gohma has 10 health, one point per sword hit, and **she is only damageable while stunned**. A stun must be manufactured: a slingshot seed into her eye while it is red, or a Deku Nut while she is on the floor. She spends much of the fight on the ceiling where she cannot be reached, and she spawns larvae.

We mined the structure from 100 recorded human victories (54,626 steps, every one a win). This is the specification of the task, measured rather than asserted **(M throughout this section, re-derivable from the corpus alone)**:

Table 4: The task, measured from 100 recorded human victories rather than asserted. Every row is re-derivable from the corpus alone.

quantity, per human win	**value**
sword hits	6.78
stuns manufactured	3.13, so about 2 hits per opening
slingshot seeds fired	2.99
Deku Nuts thrown	1.71
larvae hit	3.72
first damage at	step 190, about 19 s, a third of the episode is setup
Z-targeted	63% of steps
shield used	**0%**, the human never blocks
mean range to her eye	400 units, but **50 units at the moment damage lands**

And the gate, from 678 real damage events:

Table 5: What is true in the two seconds before her health falls, over all 678 damage events in the corpus. The engine's own vulnerability flag is set for 9 of them, which is why rewarding that flag does not work.

condition within 2 s before her health falls	**share**
she was stunned	**678 / 678 = 100.0%**
a seed impact had landed	195 / 678 = 28.8%
the engine's own "vulnerable" flag was set	**9 / 678 = 1.3%**

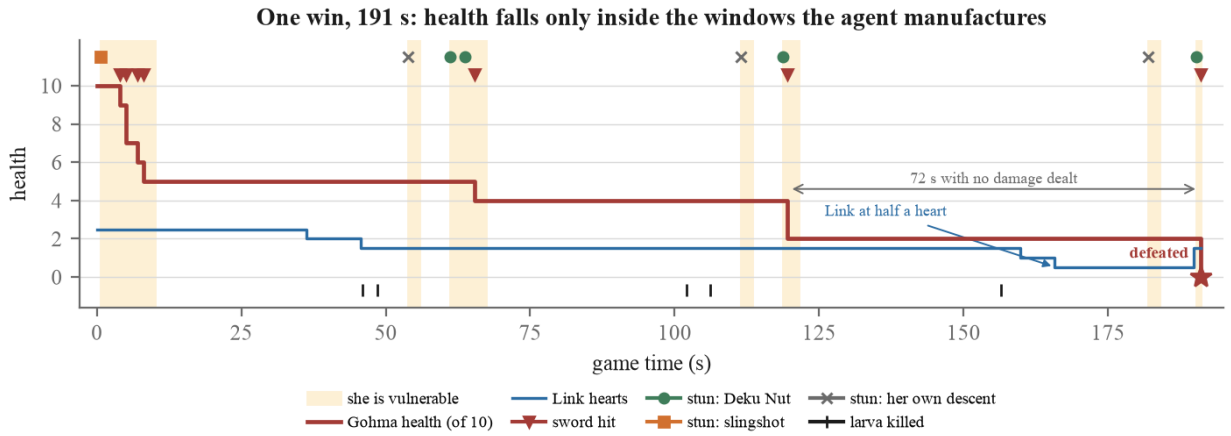


Figure 3: The anatomy of one win, 191 seconds. Her health falls only inside the shaded windows, and every window but three was opened by the agent. The longest stretch with no damage dealt is 72 seconds, during which the policy is clearing larvae, surviving and trying to re-engage. This is one episode and is shown as a mechanism, not as a rate: Figure 4 is the measurement.

The last row is worth its own sentence: the obvious flag to key a reward on identifies 1.3% of real damage events. A reward built on it would have been paying for almost nothing.

Conversion is conditional on her state at the moment of impact, across 371 impacts:

Table 6: Whether a projectile hit converts into a stun, by the state she is in when it lands. The conversion is available in one state and unavailable in the others, which is what makes the opening something the agent has to manufacture rather than wait for.

her state at impact	stun follows
falling, landing, already stunned	100%
floor, preparing	95.2%
ceiling, eye red	**40.6%**
ceiling, idle or moving	**0% across 71 impacts**

So the projectile hit that looks productive, the red-eye shot on the ceiling, converts about two times in five, and 71 of 371 impacts were wasted entirely. Hitting the red eye is necessary and not sufficient.

A disclosure about the reward we actually ran, rather than the one we designed. The boss tracker fully replaces the generic combat tracker's step function and never calls it, so the generic approach reward never fired in versions 1 through 14. Nothing in those runs paid the agent for crossing the room. We found this by reading both implementations, after fourteen versions, and the distance gradient in §5.1 should be read knowing it. The term added in v15 to fix it is the one discussed in §9.1.

Two behaviours correlate **negatively** with damage dealt across the 100 wins: larvae hit at **-0.598** and time she spends on the ceiling at **-0.559**. Fighting the larvae is what a human does when the fight is going badly. Any method that imitates the corpus uniformly learns that behaviour in proportion to how often it appears.

4.1 Why this is hard for a policy gradient, stated precisely

Generalised advantage estimation [29] forms the advantage at time t as an exponentially weighted sum of temporal-difference residuals $\delta_{t+l} = r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l})$:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \qquad \text{effective horizon} \quad H = \frac{1}{1-\gamma\lambda}. \tag{1}$$

At the settings used by every run in this project, $\gamma = 0.995$ and $\lambda = 0.95$, so $\gamma\lambda = 0.94525$ and $H = 18.3$ steps, or **1.8 seconds** at 10 Hz, against a fight that takes 45 to 154 seconds.

The consequence, asserted in a test against the real advantage computation: the terminal +60 for the kill contributes to the first decision of the episode at **0.000% of its face value**. Raising the horizon to $\gamma = 0.99967$ and $\lambda = 1.0$ brings that to 59.9% at 154 seconds.

A conflation that cost this project time is worth stating, because it is easy to make. The often-quoted "47% at 15 seconds" is the **value** discount γ^t, which at $t = 150$ steps is \$0.995^{150} = 0.47\$. What enters the advantage is the GAE weight on that same residual:

$$(\gamma\lambda)^{150} = 0.94525^{150} = 2.1 \times 10^{-4}, \qquad \text{against} \qquad \gamma^{150} = 0.995^{150} = 0.47. \tag{2}$$

Those are different numbers by more than three orders of magnitude, and only the first describes what reaches the policy gradient.

The alternative to shaping, for a terminal reward this far out of reach, is to attack the exploration problem directly with an intrinsic bonus [6] or by archiving and returning to promising states [10]. We did neither, and offer that as the obvious untried direction rather than as a result.

What that does and does not establish. It establishes that the *direct* contribution of the terminal residual to an early advantage is negligible over the length of this fight. It does **not** establish that the terminal reward never influenced earlier decisions, and the two are easy to conflate. The critic bootstraps, so information about eventual outcomes can enter $V(s)$ and propagate backwards across training updates, over many updates rather than within one trajectory. The defensible statement is narrower and still strong: **at these settings the configuration supplies almost no direct long-horizon credit, so whatever the policy learned about assembling a chain of openings had to arrive either through dense shaping inside the 1.8 second window or slowly through the value function.** The melee task is the clean case: it was solved by shaping inside that window.

5 The result: start-state dependence, not a single number

Our best boss policy is `gohma-kl-v15`, referred to below as v15. The measurement is 456 episodes per arm, 76 on each of six start states, a 300 second episode cap, and the same harness for every arm.

What is and is not held out, because the paper turns on start states and the two senses are easy to conflate. These are *fresh evaluation episodes*, played after training with no gradient updates, but from the **same six start states the policy trained on**. They are not held-out initial conditions. The four start states genuinely excluded from training are reported separately and are unusable as a generalisation test, because all four are at half health and all four beyond 368 units, so "held out" is collinear with "far" and no matched comparison exists (§10). Within-episode variation comes from sampled rather than greedy actions and from the emulator's own nondeterminism; the start state itself is a fixed savestate, so the confidence intervals describe the spread over episodes at a given start, not over start states.

The episode cap is part of the claim. An earlier evaluation used a 120 second cap, below the 154 seconds a human win takes, and so could not observe many of the wins it was counting. It reported 5.00%; the same policy at 300 seconds reads about 12% in training and 8.33% held out. Every kill rate this project measured at 120 seconds is withdrawn (§12).

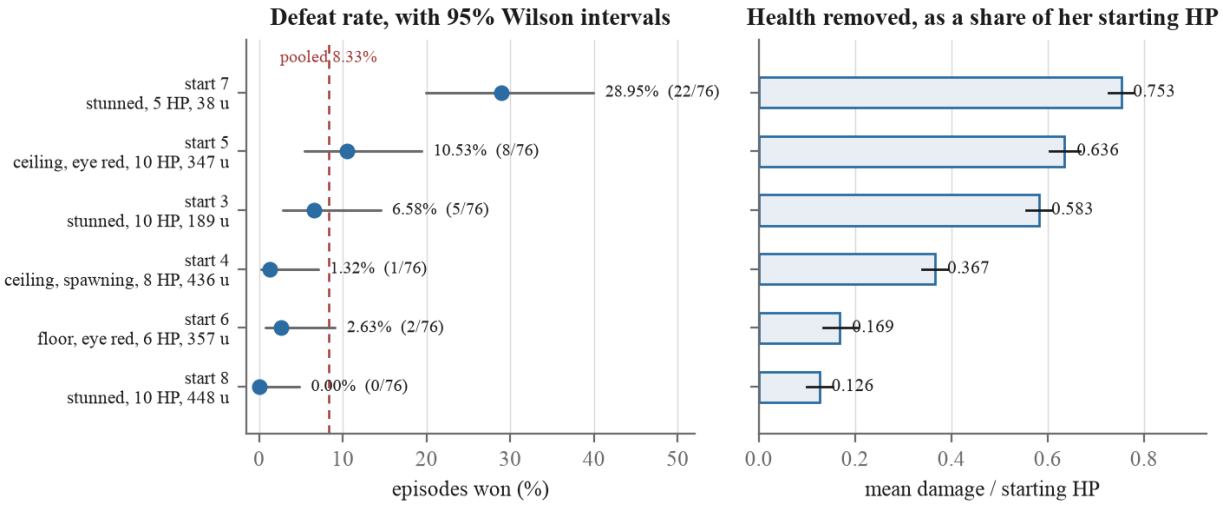


Figure 4: The same 456 episodes read two ways. Left, the binary: the confidence intervals of the four middle starts all overlap, so on defeat rate alone only the extremes separate, and starts 4 and 6 order the opposite way from the right panel with nothing to choose between them. Right, the graded quantity, as a share of the health each start hands the policy: it orders all six at a few percent of standard error. The pooled rate is an average over a curriculum we chose and moves when the mix moves.

5.1 The profile

Table 7: The per-start profile of the best boss policy, 456 episodes, 76 on each of six starts. The pooled row is an average over a curriculum we chose and is included only because results elsewhere are usually quoted that way.

start	Gohma at start	distance	wins / 76	defeat rate	95% CI	mean damage	share of her starting HP
7	stunned, 5 of 10 HP	38 u	22	**28.95%**	19.96 to 39.96	3.76 ± 0.13 of 5	**0.753**
5	ceiling, eye red, 10 HP	347 u	8	10.53%	5.43 to 19.42	6.36 ± 0.32 of 10	0.636
3	stunned, 10 HP	189 u	5	6.58%	2.84 to 14.49	5.83 ± 0.27 of 10	0.583
4	ceiling, spawning, 8 HP	436 u	1	1.32%	0.23 to 7.08	2.93 ± 0.22 of 8	0.367
6	floor, eye red, 6 HP	357 u	2	2.63%	0.72 to 9.10	1.01 ± 0.21 of 6	0.169
8	stunned, 10 HP	448 u	0	**0.00%**	0 to 4.81	1.26 ± 0.27 of 10	0.126
pooled			38 / 456	8.33%	6.13 to 11.23	3.53 ± 0.14	

(**M.** Recomputed for this paper from the 456 rows, not quoted from a log. The pooled row reproduces the figure recorded at the time, which is the check that the split is being read correctly.)

Three readings, in order of importance.

The pooled rate is not a property of the policy. It is an average over a curriculum we chose. Change the mix and it moves without the policy changing. The per-start profile is the thing that transfers, and the pooled row is included only because comparisons elsewhere in the literature are usually quoted that way.

The binary cannot resolve the middle of the table and the graded quantity can. The confidence intervals for starts 5, 3, 6 and 4 all overlap. On defeat rate alone, only the extremes separate. The damage share orders all six cleanly at a few percent of standard error: 0.753, 0.636, 0.583, 0.367, 0.169, 0.126. This is the same phenomenon as §9.1, appearing inside the headline result.

It is not one variable. Starts 5 and 6 differ by ten units of distance, 347 against 357, and by almost fourfold in damage share. What differs is her action state: on the ceiling with her eye red she presents a shootable target immediately, on the floor with her eye red she does not. Start 8 is the worst of the six despite beginning stunned, because she is 448 units away. With six starts we cannot separate distance, her remaining health and her action state, and we do not fit a model to six points.

5.2 What the policy actually learned, and what it did not

Pooling every win in the matched evaluation, 38 of them, the policy was knocked down 192 times. Of those, **136 came from a Deku Nut, 10 from a slingshot seed, and 46 from her own descent** with no projectile involved. On start 7 the slingshot converted **zero times in 22 wins**; on start 5, exactly once per win, which is the red-eye shot that start hands it.

Two things follow. First, the policy solves this boss with a different tool from the demonstrator: the human corpus fires 2.99 seeds and 1.71 nuts per win and the policy inverts that ratio. We avoid calling it the wrong tool, because the Deku Nut is a legitimate mechanic and the policy found an affordance of its action space that works. Second, and less comfortably, **24% of the openings it exploits are not openings it created.** She descends on her own schedule and the policy converts what arrives. In the recorded episode of §11 it dropped a cocked shot while her eye was red three times, at 17, 78 and 137 seconds, and passed up a fourth window at 139. It fired four seeds from the 26 it carried and only the opening one converted. Deku Nuts were already its default by 25 seconds, so this is not a policy falling back on nuts after the slingshot failed; the nut was the first tool it reached for.

This is a real behavioural difference from the demonstrations. We note it beside the imitation results of §6 as a hypothesis for why pressing the demonstrated action distribution onto this policy hurts it, and not as a demonstrated cause: state visitation, timing and recovery behaviour all differ too, and nothing here isolates item preference as the mechanism.

5.3 What each version changed, and the trajectory

Versions are referred to throughout by number. Each resumes from the previous one, which matters for reading the trend: the lineage's level change is a real measurement and the per-version slopes are not independent measurements of their own reward changes.

Table 8: What each version changed and what it did, in order. Each version resumes from the previous one, so the level is inherited and only the within-version trend is a property of the change.

version	what changed	boss damage, start to end
v10	observation v7: all larvae, velocities, facing, seed impacts	0.38 to 1.21
v11	19 shaping terms zeroed, stun credit paid only on conversion, two missing actions added	1.72 to 2.10
v12	curriculum inverted toward full-health starts	2.06 to 2.22
v13	graded health curriculum	2.41 to 2.75
v14	penalty against drifting from v13, death penalty removed	2.86 to 3.22
v15	`g_approach`, a net-progress term toward Gohma	3.17 to 3.32
v16	the same term, coefficient tripled	3.12 to 2.85
v17	coefficient back to v15's, plus deficit-weighted start sampling	3.19 to 2.86

5.4 The trajectory, and the ceiling

Across eight versions and five reward configurations, boss damage rose **0.38 to 3.32 of 10** and return rose **-12.6 to +56.8**, with every version's slope distinguishable from zero. It then **saturates at 3.2 to 3.4**: every run beginning near 2.6 climbs and every run beginning near 3.3 does not. The last six health points are the open problem, and more training time is not the missing ingredient, since one version ran 399 updates and ended at 1.21% while another ran 129 from scratch and never left 0.1%.

6 Imitation was not enough

The author beat Queen Gohma 100 times under a recorder that logs, at 10 Hz, the exact observation the policy sees, the action index the human's input decodes to, the raw controller state, and the camera yaw used for that decode. The decoder is proved to be the exact inverse of what the environment sends, round-tripping every emittable action at five camera angles; an earlier version ignored the camera and was silently wrong for every non-neutral direction.

6.1 Cloning the corpus

Table 9: The behaviour clone against the size of its corpus. Held-out next-action accuracy rises and the thing it is a proxy for does not.

	13 games	100 games
held-out next-action accuracy	53.4% (floor 11.1%)	**74.5%** (floor 20.1%)
boss damage, all starts	0.14 of 10	0.33 of 10
defeats	**0 / 80**	**1 / 80**
deaths	83.8%	81.2%
full-health starts, the actual task	**0 / 24, damage 0.12**	**0 / 24, damage 0.12**
near starts under 200 units	0.46	1.08
mid and far starts	0.00 / 0.00	**0.00 / 0.00**

Seven times the data bought 2.4 times the damage, the clone's first win ever, and nothing at all on the task. On full-health starts the two clones are identical. All of the gain is within 200 units. The single win came from the 38-unit start against a boss already at half health, which is to say it finished something it was standing on.

Event-stratified cloning, weighting each step by the decision it belongs to, is the only thing that moved the number: oversampling decision moments is monotone at matched evaluation, 0.16 then 0.33 then 0.53 of 10, and **dropping the larva class is worth quadrupling the weight** (0.49 against 0.26 to 0.33), exactly as the corpus's -0.598 correlation predicts. Across all those arms: **0 defeats in 400 episodes**. Damage scales, winning does not.

The mechanism is not mysterious, and it is not new. This is the compounding-error problem of Ross, Gordon and Bagnell (AISTATS 2011): the clone is trained teacher-forced, so every step conditions on the human's real observation history and it never learns to recover from the states its own mistakes produce, with regret that grows quadratically in the horizon. 81% of its episodes end in death. What our numbers add is the magnitude at a 154 second horizon with a demonstrator that won 100 of 100.

6.2 Frame accuracy is an anti-selector

Across five stratified cloning arms, the correlation between best validation accuracy and online damage is **r = -0.11**. Across 14 offline arms with different corpora it sharpens to **r = -0.875** (t = -6.25) on win-class accuracy, with training loss positively correlated with online damage at +0.624. The best-fitting arms are the ones with the easiest corpus.

Within a single arm, accuracy does track its own damage, weakly and positively, at r = +0.441 ± 0.097. So the same number is a usable progress signal inside a run and an actively misleading selector between runs.

And the single most compact statement of the problem. On the human's own frames, v15's greedy action matches the human's on **3%** of them. The comparison that makes that number bite is not the 0.9% chance floor over 108 actions but the **20.1% majority-action floor**: a policy that did nothing at all would agree with the human more often than v15 does. Read it as one-directional and thin. It is measured on the demonstrator's state distribution and not the policy's, with greedy actions where v15 acts by sampling, at one update of one machine, with no standard error, and we never measured the converse. Weak as it is, it is the measurement that says frame matching is a poor proxy here: the term built to raise it did raise it, from 3% to 23%, and cost damage at t = -4.98.

6.3 Fifteen mechanisms, none of which beat the policy trained without the data

Fifteen arms were run. The nine below are those that accumulated enough updates to fit a slope before the machines they ran on were lost; the remainder are listed in provenance record with their W&B identities and produced no usable trend. Every arm starts from v15's weights, so the question is whether demonstrations improve an already competent policy. The bar is v15's own 3.275 on the **mix-invariant** damage metric, by which we mean the unweighted mean of the per-start values rather than the episode-weighted average, so that changing the curriculum cannot move it. §9.2 is why that distinction exists.

Table 10: Fifteen mechanisms for folding 100 human victories into the policy gradient, each against the policy trained without them. No arm trends toward the baseline.

mechanism	updates	mean	max	slope ± SE	t
latent transition matching	17	**3.30**	4.40	-0.0120 ± 0.0331	-0.36
FitNets projection onto rollout states	19	2.61	3.85	+0.0149 ± 0.0348	+0.43
mutual alignment, contrastive	28	2.60	4.00	+0.0028 ± 0.0150	+0.19
one-way latent to the frozen clone	23	2.54	3.33	-0.0011 ± 0.0147	-0.07
clone proposes, Q head selects	16	2.18	2.95	-0.0052 ± 0.0242	-0.22
action-level BC, decaying, replicate	73	2.13	3.83	+0.0016 ± 0.0030	+0.53
action-level BC, decaying	64	2.09	3.31	**-0.0086 ± 0.0031**	**-2.75**
on-policy distillation	13	1.75	3.25	**-0.1617 ± 0.0291**	**-5.56**
IBRL	14	0.74	1.61	+0.0543 ± 0.0295	+1.84
inverse, clone as student	16	0.49	0.96	**+0.0337 ± 0.0102**	**+3.32**
v15, no human data at all		**3.275**		the bar	

This table is computed on the mix-invariant column, and that matters. These arms were originally tracked on the episode-weighted fleet mean, which moves with the curriculum. Recomputed from the exported histories on the mix-invariant column, the picture changes in both directions:

- **One arm's mean sits above the bar**, latent transition matching at 3.30 against 3.275, where the episode-weighted reading showed none above it. Its slope is flat, t = -0.36, so it is a level and not a climb.
- **Six of the ten arms produce at least one update above the bar**, against three on the weighted reading.
- **Only one arm has a positive trend distinguishable from zero**, the inverse arm at +0.0337 (t = +3.32), and it is climbing from 0.49, about a sixth of v15's level. Two arms have significant negative trends.

And none of it is a measurement, which is the part that matters. Every one of these histories is a **single machine**. Per-machine balanced damage on this task spans **1.36 to 3.37** at identical settings, so a one-machine mean of 3.30 sits comfortably inside the spread of one configuration run twice. The bar itself, v15's 3.275, comes from a different aggregation and was never re-derived on this column. The defensible statement is therefore weaker than either reading: **across ten mechanisms, none shows a trend toward or above the policy trained without demonstrations, and the design cannot resolve a difference the size of the one arm that nominally exceeds it.** The machines were lost before any checkpoint was synced, so these arms cannot be re-evaluated, only re-read.

A cleaner decomposition came from a three-arm, three-seed study on the policy's **own** online states. Aligning only the internal representation to the clone is neutral on every seed (+0.032, -0.066, +0.123). Adding a penalty on the clone's **actions** at those same states is harmful and not marginally so, at -0.107 (t = -5.50) and -0.110 (t = -4.83). Pressing a 0.53-damage policy's action distribution onto a 3.275-damage policy pulls the latter toward the former, which is the obvious thing to say afterwards and was not obvious before it was measured.

This is the **imitation gap** named by Weihs, Jain and colleagues (ADVISOR, NeurIPS 2021): when the teacher acts on information or capability the student does not have, imitating it is actively harmful and can be beaten by pure reinforcement learning. Our instance has two unusual features, set out in §14: the demonstrator is not privileged, since the human sees exactly the observation the policy sees, and the demonstrator is far stronger at the task rather than a crutch for a weak learner.

6.4 What did work offline, and what it says

Return-conditioned behaviour cloning conditions a cloned policy on the return it should achieve and reads the action out at test time by asking for a high one. It is the "RvS" formulation [11], the non-sequence-model

member of the family that Decision Transformer [7] made standard. Trained on the same 100 human wins **plus** a few hundred of the policy's own failures and 200 of its wins, with no policy gradient anywhere, it reaches **1.76 ± 0.11 damage and 8 defeats in 240 episodes (3.3%)**. Both plain clones of the same 100 games scored **0 in 400**.

These were measured at a 120 second episode cap, which §5 says is below the time a human win takes and which invalidated an earlier headline. The damage figures are usable because damage accumulates throughout an episode; the defeat rates in this subsection are not comparable with the 300 second numbers anywhere else in the paper, and v15's own damage moved from 2.88 to 3.53 between the two caps. The one comparison below that is matched at 300 seconds is labelled as such.

The informative contrast is the corpus, not the architecture:

Table 11: Offline return-conditioned cloning by corpus composition, pooled over every evaluation rather than read off the last one.

corpus	pooled damage	defeats
human wins + failures + 200 policy wins	**1.76 ± 0.11**	3.3%
human wins + failures	0.93 ± 0.17	about 1%
human wins alone	**0.30 ± 0.04**	0.4%

Training on victories alone is the worst configuration by a wide margin. The failures are what label the fatal action. Adding the policy's own wins to the win class is worth +0.67 ± 0.20 (t = 3.3), and more of them is not monotonically better: 200 beats all 495 at t = +2.43.

It still loses to the policy gradient. At matched 300 second caps, 456 episodes per arm, same harness:

Table 12: The best offline arm against the policy gradient, at matched episode caps and the same harness.

	v15, the policy gradient	best offline arm, return-conditioned cloning with model-generated data
defeats	**38 / 456 = 8.33%**	18 / 456 = 3.95%
starts won from	**5 of 6**	2 of 6
boss damage	**3.53 ± 0.14**	1.31 ± 0.08

Two-proportion z = +2.76 on the binary; **t = +13.91 on the graded metric**. Off the easiest start, v15 averages **3.48** health to the offline policy's 0.88, a factor of 3.95 (recomputed from the 760 non-start-7 rows; our provenance record carried 4.26 and "4.8x" until it was recomputed, which was wrong). Note the ratio of evidence: the graded quantity separates these arms at 14 standard errors where the binary manages 2.8, on the same episodes.

Two further offline results are worth recording because they close plausible escape routes. Corpus size from 800 to 1,800 episodes at fixed composition is worth **+0.17 ± 0.14**, indistinguishable from zero, while ten times the episodes at a diluted win fraction is **worse** than two times at fixed composition. And a bug worth generalising: sharding the 100 demonstrations across seven machines, 14 each and disjoint, produced a 3.4-fold spread in win-class accuracy across machines and a collapse at -0.0639 ± 0.0109 per epoch. Keeping the scarce class whole on every machine and pinning its share of the gradient cut that to -0.0165 ± 0.0083, a difference of +0.0474 ± 0.0137 at t = +3.46. The mechanism was real and the fix was insufficient: the repaired run still does not rise.

7 The world model did not get far enough

The motivation is the standard one: a model accurate enough to act in lets a policy be improved on model-generated experience rather than on environment steps [32, 15, 16], an approach that now carries the state of the art on many control benchmarks [17] and that can be used for search when the model's value is reliable [28]. Our closed loop in this section is the Dyna pattern at small scale, and the rollout depth it should have used was recorded in our own earlier work on it [19].

Four phases over two months, each with its own architecture family, dataset scale and reason for being abandoned. The baseline throughout is **persistence**, the prediction that nothing changes, and **skill** is `1 − model error / persistence error`: zero means exactly as good as doing nothing, positive is better than doing nothing, and negative is worse. Every number in this section is a skill score in that sense.

The baseline is not ours. Copying the last frame is long established as the floor in video prediction, where it is known to beat learned models on mostly static data [34, 33], and a NeurIPS 2026 workshop paper, "Your world model may be worse than copying the last frame" [31], reports that of 20 recent world-model papers, 12 give a prediction-quality number and 9 of those give nothing to read it against. This section therefore confirms a known and under-reported failure mode rather than discovering one; §14 states the narrower thing we think is ours.

Table 13: The world model by phase, and what each phase was measured against. The trivial control is copying the last frame.

phase	scale	verdict
latent transition models, 6 configurations	203k transitions	every one **loses to persistence**, best -1.14, worst -7.24
object-centric causal model	same	the adopted paper's actual method was never run; what trained was its baseline
entity tokens, fleet scale	**20.5 M** transitions	**validation loss fell 1.8 times while rollout skill collapsed from +0.003 to -0.41**
tokens plus history, single H100	**91.5 M** transitions, resident in VRAM	positive through 5 steps; the 1800-step figure is not a finding, see §10
boss observation model, 1024 wide	4.7 M transitions, 120 epochs	**getting worse at -0.0874 ± 0.0218 per epoch, t = -4.00**

(**M** for every row. The first three rows are Deku Baba models on Deku Baba data; only the last is the boss. The three diagnostic findings below are Deku Baba measurements and do not describe the boss model. **The last row is not the model behind the planning-loop result later in this section**: that loop trained six separate and much smaller models of its own, and none of them is in this table.)

The last row is the clearest statement available. After 120 epochs, with the output layer zero-initialised so that the model **begins** exactly at persistence, 120 epochs of training moved it steadily away from doing nothing. Its event heads, pushed to catch rare damage events, learned to fire on almost everything: 930 to 950 false events per 1,000 quiet ticks at precision 0.003 to 0.009. And the decisive control:

Shuffling the action changes the prediction by a factor of 1.00. The action is decoration.

Said precisely: under the action-shuffle control, at the horizon evaluated, the model's predictions are effectively invariant to which action it is given. That is an aggregate over states, and our own counterfactual

measurement below says 43% of states are action-insensitive in the real game, so the aggregate will understate conditioning that exists only in the other 57%. We did not run the shuffle control restricted to the action-sensitive subset, and that is the measurement that would sharpen this. What the aggregate does support is that nothing can be planned in this model as it stands, since a planner's leverage is exactly the difference between actions.

A closed real-games to model to dream-policy to real-evaluation loop ran nine iterations on six machines, each machine training its *own* small 256-wide observation-space world model on its own freshly collected games rather than the 120-epoch model above; every dream-trained candidate won **0 of 607 real episodes**, with damage flat to declining at -0.056 ± 0.033 per iteration. The two failures are separate measurements of the same trainer and the same event weighting, and the loop's own per-iteration world-model readouts were lost with the pods, so the action-shuffle result is the mechanism we propose for the loop's failure, not a measurement of it. The loop's models were 256 units wide against the fleet model's 1024 and saw 60 freshly collected episodes per iteration for 2 to about 18 epochs, against 2,605 recorded episodes for 120 epochs. Two caveats attach to the 607: only **50 of the loop's 54 evaluations were recovered**, so the per-iteration n is 55 to 76 rather than the 120 quoted elsewhere in this report, and the loop's evaluations ran at the **120 second episode cap that §5 and §12 withdraw**, so the candidate-versus-anchor comparison within the loop holds but the anchor rate it was compared against does not. The loop's machinery is validated; its model is not.

Three diagnostic findings generalise beyond this game:

- **Motion is not action-conditioned at one step.** Ablating the action costs +0.002 of skill, and the cosine between the commanded action and the resulting displacement is 0.006 at lag 1.
- **43% of states are action-insensitive**, measured directly with 150 counterfactual pairs: identical state, two different movement actions held for three steps, both real outcomes recorded. The other 57% separate by a mean of 21.4 world units. Averaging over both populations is what makes action-conditioning look uniformly absent.
- **The objective mattered more than the architecture.** Changing the loss from squared error to absolute error, with the same network, data and epochs, moved aggregate skill from **-0.379 to +0.392**. And a 1.35M-parameter MLP matches both attention models at one step, so one-step skill does not discriminate architectures at all.

8 One representation result that clears its controls

The four earlier attempts at self-supervised representation learning here all optimised the same objective: predict the next latent from the current latent and action. All four lost to the identity, for a mechanical reason that was eventually measured: the latent moves 0.023 to 0.05 of a random pair's distance per step, so copying is near-optimal.

The masked-prediction objective of I-JEPA [3] and V-JEPA [4] does not pose that task. It masks part of the **input** and predicts a target network's representation of the masked part from the visible remainder. There is no persistence shortcut because there is no previous latent. **That objective had never been run here.** Masking over decision data is not ours either: MTM (Wu et al., ICML 2023) already trains one network to reconstruct trajectories of states and actions from random subsets of themselves. What we claim is the masking structure and the free-signal floor, and §14 says why.

We ran it on the raw 363-field structured state, not the featurised policy observation, because the latter is dense with derived redundancy: masking a field whose inputs remain visible scores arithmetic rather than understanding. Masking is structured in the same spirit as V-JEPA's multi-block scheme, translated to time by entity by field: a whole actor removed for a whole clip, a whole kind of field removed for every actor, a time block, or scattered cells. Achieved mask ratio 0.791.

Then the encoder is frozen and probed. The probe reads a **masked** encoding and predicts the **hidden** content, because probing an unmasked state for a field that is an input to it scores the identity map. There are three floors to clear, and they are the three columns before the model's own in the table below: the **trivial floor**, which is always predicting the majority class or the mean; the **free-signal control**, a ridge regression fitted on the cells that are still visible, which matters because a masked cell is often exact arithmetic on a visible one; and an **untrained network of identical architecture**.

Table 14: Frozen probes on the masked-prediction encoder, against three floors. An entry beats its floors or it does not count; the untrained column is the same architecture with random weights.

probe	trivial floor	free-signal control	untrained, same architecture	masked model
which actor is locked on (accuracy)	0.496	0.718	0.714	**0.919**
melee animation (accuracy)	0.327	0.593	0.603	**0.798**
action routine (accuracy)	0.378	0.603	0.627	**0.791**
enemies alive (mean abs. error)	0.772	0.539	0.565	**0.181**
nearest enemy distance (world units)	74.3	71.7	93.1	**44.1**
floor height (mean abs. error)	4.16	3.11	4.54	**2.08**

All ten scored probes beat both an untrained network of identical architecture and a ridge regression on the cells still visible. One of the ten, a background-flag classifier, ties the majority-class floor to three decimals and clears the other two by under one standard error, so we strike it by hand and report **9 of 10**. Five further probes are excluded outright because their target is constant in this dataset. Counting is the strongest surviving result: 0.181 average error on how many enemies are alive when every cell that would tell you is hidden, against 0.539 for the free-signal control. This is the first representation in this project not beaten by random weights; the previous attempt was worse than untrained on 7 of 10 fields.

What it does not establish, stated in the same breath. **One seed**, where six seeds of one configuration have previously spanned a factor of 2.5 here. The objective **still loses to its own trivial control**: emitting an unrelated clip's representation scores 0.0273 against the model's 0.0815, so it is improving toward that control, from 8.1 times to 3.0 times, and has not passed it. The probe is **not capacity-matched**: the encoder arm is given 13,056 features and the free-signal control 7,344, so the winning arm has 1.8 times the width to regress on, and a single ridge constant is applied across arms whose features are layer-normalised in one case and about three-quarters zeros in the other. A per-arm sweep of that constant has not been run. The low-shot axis did not run at all: a floor on sample count collapsed two fractions to the same number.

Two deviations from the papers, shipped in the implementation rather than proposed. I-JEPA's context encoder processes only the visible patches and V-JEPA removes the masked ones, putting learnable mask tokens in the **predictor**. We fed per-field mask embeddings to the **encoder**, which is the masked-autoencoder arrangement rather than the JEPA one, and used 54 distinct mask vectors where the papers use one shared vector plus a positional embedding. The result above is a property of what we ran, not of the published method, and a paper-faithful arm has not been run against it.

9 The instrument was the hardest part

This section is the one we would keep if we could keep only one. In this project, measurement error exceeded method error.

Most of what follows confirms rather than discovers. Henderson et al. (AAAI 2018) and Agarwal et al. (NeurIPS 2021) established that few-seed point estimates mislead, that aggregates need interval estimates, and that conclusions flip when this is done properly; our measured noise floors, our winner's curse and our selector arithmetic are instances of their argument with the specific wrong decision each one caused attached. §14 marks the two parts we have not found stated elsewhere.

9.1 A sparse binary cannot steer a task like this

Every "is it improving" answer came from the defeat rate, a binary at 1 to 10%. On that signal, six consecutive versions read flat and two experiments were launched to break the apparent plateau. On boss damage, the graded quantity the reward principally pays for, the same runs read:

Table 15: Boss damage and return across eight versions, read as first-third and last-third means. This is the same data every status report was read through as a defeat rate, which is Figure 5.

version	boss damage	slope	return
v10	0.38 to 1.21	+0.003 *	-12.58 to 15.46
v11	1.72 to 2.10	+0.015 *	1.06 to 18.75
v12	2.06 to 2.22	+0.008 *	4.87 to 6.63
v13	2.41 to 2.75	+0.007 *	18.93 to 26.36
v14	2.86 to 3.22	+0.012 *	45.54 to 52.62
v15	3.17 to 3.32	+0.004 *	54.25 to 56.79
v16	3.12 to 2.85	**-0.017** *	56.08 to 46.88
v17	3.19 to 2.86	**-0.022** *	52.35 to 37.55

$*$ marks a slope distinguishable from zero. **The project improved 8.7 times with every step statistically solid and it was reported as six consecutive failures.**

What the table is computed from, and what Figure 5 is computed from, because they are not the same artefact. The rows above were produced while the fleet was alive, from the pods' own training logs, pooled across the ranks of each run. Those pods are gone. What survives is the exported run history for each version, which is the aggregator rank alone, and the figure is drawn from that so a reader can re-derive it. The two agree on everything the argument uses: the same direction on all seven versions the export covers, the same two regressions, the same ordering, and first-third and last-third means within 0.15 of the quoted endpoints. They disagree on per-version significance, and in the conservative direction: on one rank rather than pooled ranks, three of the seven clear `|t| > 2` instead of all of them. The 8.7-fold claim spans v10 to v15 and v10 predates per-slot logging, so v10 is in the table and not in the figure. Readers who want the pooled form have the table; readers who want to re-run the arithmetic have the figure and the exported histories. A defeat requires ten successful openings; damage counts each one, so one signal can show a trend on a few hundred episodes and the other cannot.

The wrong metric then caused harm rather than merely hiding progress. An approach-shaping term, `g_approach`, was added in v15 on the strength of a defeat-rate band analysis. The last two rows are the versions that tripled its coefficient and that added deficit sampling, and both are regressions against the policy they resumed from, confirmed on the mix-invariant per-start form (t = -4.75 and -3.78) so neither is a

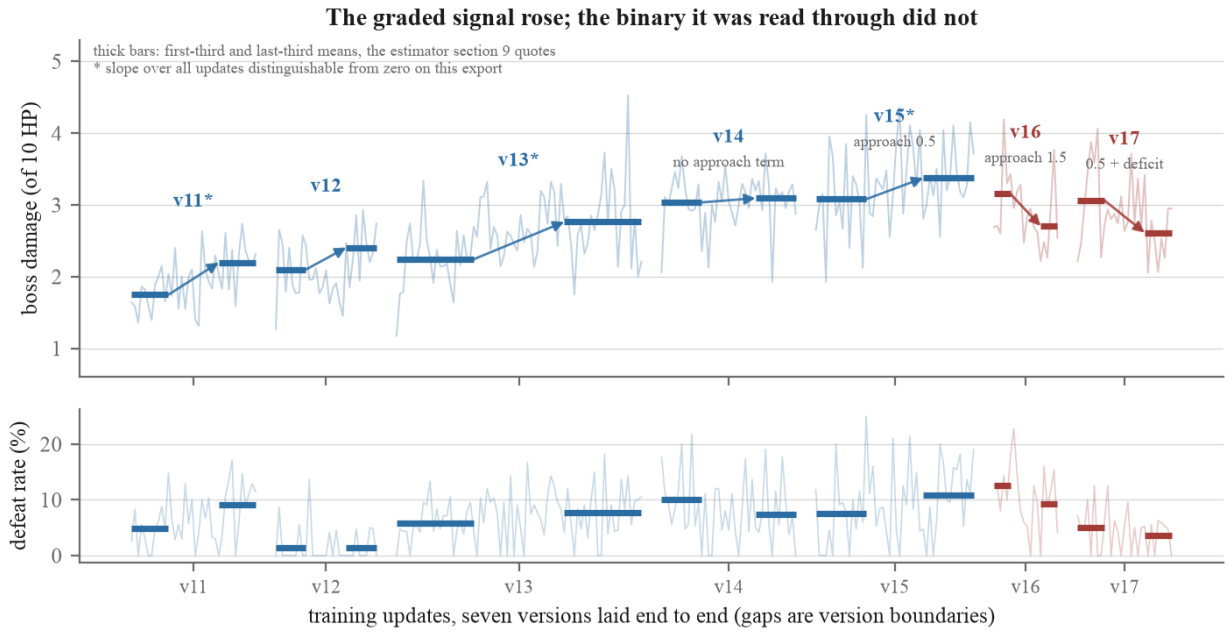


Figure 5: Seven versions laid end to end, the same runs read through both instruments. Above, boss damage, the graded quantity the reward principally pays for. Below, the defeat rate, the binary every status report was read through. Thick bars are the first-third and last-third means, the same estimator in both panels. The graded signal climbs across the first five versions and falls on the last two; the binary, on the same data, is noise at this episode count. The two versions drawn in red are the ones added to fix a plateau the binary appeared to show.

curriculum artefact. **We stop short of saying the term caused the plateau**, which we did assert internally before checking: the slope also tracks each version's starting level, and a later arm that removed the term from the same weights did not recover the earlier slope. Our own claim audit marks that causal reading confounded. What is attributable is narrower and still useful: two changes motivated by the sparse binary were measurably harmful, and the saturation at 3.2 to 3.4 is a property of the task across eight versions and five reward configurations.

The rules this produces. Rank candidate metrics by how well they separate the comparison you actually want to make, not by their name. The binary responds only when the whole ten-hit chain completes, so every partial improvement is invisible to it, while the graded quantity moves with each hit. The claim is about the metric's response to the underlying change and not about its spread, since larger variance is not by itself more information. The empirical form is the one to quote: on the same 456 episodes per arm, boss damage separates our two best methods at t = 13.91 where the defeat rate manages z = 2.76. Make the headline the quantity the reward's largest terms pay for. Report a trend with its standard error, because "flat" and "rising" are claims about a slope. And never let a sparse metric justify adding a reward term.

That last rule has a theoretical companion. Ng, Harada and Russell (ICML 1999) proved that only a shaping term of the form `gamma * Phi(s') - Phi(s)`, a difference of a potential over states, leaves the optimal policy unchanged; any other additive term can change what is optimal. None of our shaping terms, `g_approach` included, is of that form, so the reward we ran is not policy-invariant by construction and a regression from adding one is the expected failure rather than a surprise. §14 gives the measured price of the same mechanism in an earlier version.

9.2 A metric that moves with the sampling distribution cannot compare policies

The defeat rate is episode-weighted over start states we choose. Version 11 showed the best trend of any version, +0.270 per update over its final third, and was stopped early to try something else on that basis. Its per-start history:

Table 16: Version 11's per-start history, which the pooled rate concealed. The aggregate climbed because the curriculum weighted the one start it could win at 50 percent.

start	distance	first third	last third
7	**38 u**	7.32%	**14.00%**
3, 6, 2, 8, 5, 1, 4	188 to 541 u	0.00%	0.00%

It was not learning the boss. It was learning the one start where Link begins on top of her, and the aggregate climbed because that run weighted that start at 50%. Nine starts sat at exactly zero for 37 updates.

The same instrument then lied in the other direction. Version 15 reads as the flattest run in the table while being the first version non-zero from **6 of 6** trained starts where v11 was non-zero from 1 of 10, and about 25 times broader off the easy start.

Two consequences. Any headline that averages over a distribution you control must be reported next to a mix-invariant form of itself, and the mix-invariant form is what a version-to-version claim cites. And, less obviously: the policy gradient optimises expected return **under** the sampling distribution, so the curriculum weights are part of the objective, not part of the evaluation.

9.3 Three versions of one mistake: an n too small to resolve what it is selecting on

The same error appears at three levels, and we made it at all three.

At the level of a checkpoint. A held-out evaluator chose which checkpoint to keep, ranking on 16-episode subsets. The pooled rate over every evaluation was 1.68%. At that rate a 16-episode subset reads 0.00% **76.2%** of the time and 6.25% **20.9%** of the time, so the 6.25% that crowned a checkpoint was one win in sixteen, which any checkpoint produces one time in five. Tested directly, the selected checkpoint against every other gives p = 0.094. Separating a genuinely twice-as-good checkpoint needs about 256 episodes each, sixteen times what it had.

At the level of an arm. A 12-cell grid ranked on roughly 40 episodes per cell produced a winner reading +32% over control. Confirmed as two matched arms from one initialisation, the gap is **+0.065 ± 0.117, t = +0.56**. The entire +32% was the upward bias of taking the maximum of twelve noisy estimates. The grid's qualitative finding, that long credit horizons need low learning rates, is monotone across all twelve cells and survives; the winning cell is fiction.

At the level of the pool. Writing a script to pool evidence, replacing analysis that had been retyped per question, exposed three errors in the hand-pooling, all of which had moved p-values toward significance: 2,831 episodes from a withdrawn evaluator were still in the pool; the same evaluation existed in three copies, so one arm pooled to 1,551 episodes where it ran 880; and the quoted baseline was a 559-episode subset of an 880-episode run. Correcting all three moved a comparison from p = 0.122 to p = 0.087 on the same data.

A fourth variant, one level lower again: the newest point of a fleet's evaluation curve is usually **incomplete**, because machines finish at different times and the ones that finish first are not a random subset. One point read 1.105 on two machines and 1.250 on six. Three consecutive slope claims were made and reversed from that in forty minutes.

The rule we now apply. Before a statistic selects anything, compute what it reads under the null. If the winning value is a likely draw from the null, there is no selector, only a lottery with a percentage printed on it. State the sample size the selector needs and compare it to the one it has.

9.4 What a claim has to clear here

Four independent measurements of the noise floor, all on identical configurations:

Table 17: The noise floor, measured four independent ways. Any claimed effect smaller than these is not distinguishable from rerunning the same configuration.

replicate set	spread
five identical policy-gradient controls	2.00 to 2.97, sd 0.380
two identical offline seeds	0.43
two groups of three identical runs	differ by 0.25 ± 0.12, **t = 2.1**
six seeds of one world-model configuration	trajectory error 1.38 to 3.43 times

The third row is the useful one: that reading crossed the usual significance bar while being pure seed noise. A three-seed group mean carries ±0.1 here, so nothing below about 0.3 damage is visible with that design. Two offline seeds at identical settings produced **opposite** significant slopes, +0.032 (t = 2.2) and -0.030 (t = -2.4), which is the plainest available statement that one seed's slope is not a measurement.

9.5 A number read out of a post-mortem instead of a recommendation

The world-model result of §7 was run at a rollout depth of 600. The author's own earlier Dyna work, the code and manifests of which were available throughout, trains at a depth of **10**, and that setting is recorded in its checkpoint manifest where one command would have found it. The price of the discrepancy is measured: at depth 600 the model degrades at **-0.1675 ± 0.0562 per epoch (t = -2.98)**, and at the reference's depth of 10 it sits at persistence instead.

Where 600 came from is the part worth generalising. It appears in an earlier write-up of our own, in a passage describing a horizon mixture that had been **measured as a regression**. A number quoted inside a post-mortem is a record of what failed, not a setting to adopt, and we read it as the latter. Two rules follow, both about the instrument rather than the method: ask for the reference implementation of a method before building it, because a manifest answers depth, discount and batch size for free; and when taking a hyperparameter from a document, take it from what that document concluded, not from its narrative.

10 Limitations and threats to validity

- **One seed** for the masked-representation result, and for several world-model comparisons. By this project's own measured floors that is below the resolution needed.
- **Start-state coverage.** The boss curriculum trains on six of ten starts, and the four held out are all at half health and all beyond 368 units, so "held out" is collinear with "far" and no matched comparison exists. We therefore make no generalisation claim.
- **Damage pooled across starts with different starting health.** The pooled 3.53 "of 10" mixes starts at 5, 6, 8 and 10 health. The share-of-starting-health column is the comparable one.
- **The melee per-state result is one watched episode per state**, so it is qualitative and is not a rate.
- **The wall map used by the world model covers 209 of 1,024 cells**, only where the agent happened to collide with something.

- **The long-horizon world-model numbers are thin**, which is why §7 does not present them as a finding: the 1800-step skill of -2.44 rests on about 31,000 of 52 million possible starts qualifying at that horizon, and all of them are timeout episodes by construction.
- **Reproducing on macOS requires a code-signed interpreter** to attach to emulator memory. Linux does not.
- **The masked-representation substrate exists on one machine.** The 203,822-frame raw-state datasets behind §8 are not mirrored off-site. The checkpoint is durable and its input is not, so that result is currently reproducible only from local disk.
- **No weights exist for any of §6.3's fifteen arms.** The machines they ran on were lost before any checkpoint was synced, so what survives is one machine's training history per arm. That table cannot be re-evaluated, only re-read.
- **The row file behind the 13-game clone's 0 of 80 no longer exists**, on this machine or in remote storage. Only the aggregate training-run history remains, so that cell rests on thinner evidence than the table implies.
- **§6.4's offline figures were measured at a 120 second episode cap** and the rest of the paper at 300. The damage numbers survive the difference; the defeat rates in that subsection do not transfer.
- **No architecture ablation was run anywhere.** The entity-factored recurrent policy was never compared head to head against a flat network at scale, so §2.3 describes a choice and not a finding.
- **The per-step observation latency figure has no artefact.** We quote it from development notes.
- **Several headline comparisons mix within-run slopes with between-arm levels.** Where we noticed, we say so in place; where we did not, treat a slope as evidence about one run's trajectory and not about the ranking of two methods.

11 Media

Both clips are the real emulator at 10 Hz control, cropped to the game area, with game audio. Unedited, as played:

- **Queen Gohma, full fight:** `https://youtu.be/FBD1wEsdY3E` (3 min 11 s)
- **Deku Babas:** `https://youtu.be/fMMvd1HmMjI` (27 s)

The melee task, end to end (`media/deku-slot1/deku-slot1_s01_try01_crop.mp4`, 29.7 s, md5 `d20c62aae595`). The agent starts just inside the room entrance at full health with all three Babas underground 530 to 830 units away, walks up, wakes them and clears the room. Kills at 7.3, 22.4 and 26.8 s, 1.5 hearts lost. First attempt.

Queen Gohma from the full-health start (`media/gohma-slot5/gohma-slot5_s05_try04_crop.mp4`, 197.7 s, md5 `a184dbd29233`). She begins at 10 of 10 health on the ceiling with her eye red, 347 units away. The fight, from the episode's own log:

Table 18: The recorded episode, moment by moment. One episode, shown as a mechanism rather than as a rate.

time	what happens
0.6 s	slingshot seed into the red eye, she falls. **The only converted seed shot of the episode.**
4.0 to 8.0 s	five sword hits inside that opening, 10 health down to 5
45.9, 48.6 s	she is back on the ceiling; two larvae killed with the sword
53.9 to 65.4 s	she falls, two Deku Nut stuns, one hit, 5 down to 4
102.0, 106.2 s	two more larvae killed
111.4 to 119.5 s	falls again, nut stun, two hits, 4 down to 2
156.5 s	two larvae killed with the sword on the same tick
181.9 to 191.0 s	she descends a third time unprompted, nut stun, two hits, **defeated at 191.1 s**

Ten sword hits and eight knockdowns, **of which the policy caused only five**: one from the slingshot and four from Deku Nuts. The other three were her own descent, at 502, 644 and 688 units away, with no seed or nut involved. Thirteen larva hits and **six kills**, two hearts lost, Link down to half a heart at one point. Six is a correction: the clip manifest printed eight and the per-tick log printed five, and both numbers were wrong, for separate reasons we have since fixed. The manifest summed downward steps in the HUD's alive-larva count and called each one a death, which counts two larvae that left the room alive, one untouched at 90.4 s and one at 1 of 2 health at 179.9 s, neither with a damage event or a kill reward paid. The log **assigned** its kill event where it accumulates its hit event, so the two larvae that died on the same tick at 156.5 s emitted one event between them. Three independent channels force six: 4 points of damage on that tick cannot come from one larva holding 2 health, the alive count falls from 3 to 1 across it, and the return rises 5.98, which is 4 hits at 0.5 plus 2 kills at 2.0. Total larva damage closes exactly at 13, six larvae at 2 health plus the single point taken off the one that despawned alive. **The per-actor episode counter was correct throughout**, and it is what the matched-evaluation rows carry, so nothing in the quantitative results is affected.

It took four attempts, and the three that failed are in the manifest: deaths at 2, 5 and 2 of her 10 health. That start wins 10.53% of the time (8 of 76 matched episodes), so four attempts is a slightly lucky draw on the attempt count and an entirely ordinary win. Every one of those 8 matched wins also kills larvae and takes all ten health, so the clip is representative **of a win at that start**, and the attempt count is what says how often a win happens.

Both directories carry an `attempts.json` recording every attempt, won or lost, with each clip's duration checked against the episode's own length and a frozen-frame fraction confirming the capture is live gameplay rather than a still. Three separate capture faults were found and fixed to get these, including one that silently filmed an empty desktop while passing a duration check; they are recorded in provenance record section 17.

12 Retractions

Claims this project made internally and then overturned. These are corrections to our own working claims, not to anything published elsewhere. They are listed because the correction is the reusable part, and because a paper arguing that claims outrun evidence should show its own.

Table 19: Every claim this project published internally and then withdrew, with the control that killed it. The corrections are the reusable part and are kept rather than deleted.

what we claimed	what killed it	where
A competent policy's latent is an excellent prediction target, skill +0.742	Random untrained weights of the same architecture scored +0.617. Removing the copy of the previous action took it to +0.240, velocity to +0.053, a countdown and episode constants to -0.012. The whole number was artefacts	7
Version 11 is the fastest-improving version, +0.270 per update	One start state at 50% curriculum weight; nine starts at exactly zero for 37 updates	9.2
Six versions plateaued	They rose 8.7 times on the graded metric	9.1
The 9-second, 3e-5 grid cell beats our settings, +32%	Maximum of twelve noisy estimates. Matched confirmation gives +0.065 ± 0.117	9.3
v15 defeats her on 5.00% of episodes	Measured at a 120 second cap, below the 154 seconds a human win takes. Every kill rate at that cap is withdrawn	5
Offline return-conditioned cloning matches the best policy gradient at killing	Both sides were at the short cap. At matched 300 second caps the policy gradient wins by 14 standard errors on the graded metric	6.4
Four machines trained one sharded world model	They averaged nothing on any round: the aggregator was dead from launch and the reduction crashed on a device mismatch. Four independent quarter-corpus models. Visible all along in 616 second sync epochs against 15 otherwise	7
Three world-model conclusions	Artefacts of the instrument: scoring a gate on a quantity the gate excludes; measuring a grid by re-fitting rather than reading the stored one; a threshold rule derived from a normalisation bug	7
The masked-representation run should be killed	Recommended twice at five epochs from +0.0026 ± 0.0030, read as flat. At twelve epochs it is +0.0150 ± 0.0015, t = +10.29. The arithmetic was right and the number was not decision-grade	8
The approach term caused the plateau	Confounded by each version's starting level, and a later arm removing the term did not recover the earlier slope. It survived into a draft abstract of this report before review caught it	9.1
Off the easiest start v15 averages 4.26 health	Recomputed from the 760 rows: 3.479. No subset of starts yields 4.26. The error was in our provenance file and the paper inherited it	6.4

13 Reproducibility

Every number above traces to a directory. provenance record carries the resume lineage per version, wall clock and cost, the step-axis discontinuities that make a curve misreadable without them, and why each version exists including the ones that failed. claim audit is one row per claim with its evidence class. `artifacts/MANIFEST.json` identifies every checkpoint, dataset and result file by md5 rather than by filename, with its remote key where one exists, because a stray checkpoint file cannot otherwise be matched back to the run that produced it.

Training histories are **exported, not linked**: each run group has a local directory with its configuration, summary and full history, including the identities of all per-machine runs so a fleet is recoverable and not just its aggregator. In practice one group's per-machine histories were not pulled before the machines were

lost, which is the gap behind §6.3.

The commands, so the two results a reader is most likely to want are runnable rather than described:

```
# watch the solved melee policy play, in a real emulator window
DOLPHIN_BIN=~/.cache/zelda-rl/Dolphin.app/Contents/MacOS/Dolphin \
  .venv/bin/python-dbg scripts/play.py checkpoints/kept/deku-combat-gru-v2b-20p_u00120.pt \
    --slots-dir datasets/zelda/combat/deku-baba/states --slot 1

# the masked-prediction result of section 8, end to end on CPU, about 35 minutes
ZRL_ALLOW_NO_WANDB=1 .venv/bin/python -u scripts/train_sjepa.py \
  --data 'datasets/transitions/v3-main' --data 'datasets/transitions/v3-moves' \
  --data 'datasets/transitions/v3-approach' --data 'datasets/transitions/v3-scenarios' \
  --out runs/sjepa/p1.pt --prep configs/sjepa_prep_v3.json --fit-prep \
  --clip-len 8 --stride 12 --mask-ratio 0.75 --dim 96 --depth 3 --heads 4 --pred-depth 2 \
  --batch 256 --lr 1e-3 --epochs 12 --torch-threads 6 --seed 0 --fresh --run-name p1

.venv/bin/python -u scripts/probe_sjepa.py --ckpt runs/sjepa/p1.last.pt \
  --data 'datasets/transitions/v3-main' --data 'datasets/transitions/v3-moves' \
  --data 'datasets/transitions/v3-approach' --data 'datasets/transitions/v3-scenarios' \
  --limit-clips 2500 --stride 16 --feat grid --min-n 200 --out results/sjepa/p1_probe.json
```

One warning for anyone pulling artefacts: four world-model checkpoints exist in remote storage under the same filenames as the ones cited here but at roughly a third of the size. They are an earlier run. Match by md5.

14 Related work and positioning

Citations are verified and annotated in the literature file, which also lists the clusters not yet covered. We state what is not ours first, because the honest version of this section is mostly that.

Reading engine state instead of pixels is not new, and the nearest precedent is close. Firoiu, Whitney and Tenenbaum [13] trained agents for *Super Smash Bros. Melee* under emulation, reading each player's position, velocity **and action state** from game memory every frame, and beat human professionals. That is the same emulator family, the same commercial console game, and the same decision we made about the observation, including the decoded action state that makes an enemy's intent legible. Our setting differs, a single-agent 3D boss encounter with a chain of manufactured openings over 45 to 154 seconds rather than a reactive two-player fight, and our interest is in what fails to improve a policy there. We claim the setting and the measured task specification, not the observation approach.

Where this sits among agents that play commercial games. The results this field is known for are at a scale we are not working at: AlphaStar [35] and OpenAI Five [24] reach professional human play on commercial titles with tens of thousands of years of experience, and the open-world Minecraft line [14, 12] studies human demonstration corpora orders of magnitude larger than our 100 recorded wins. Deep RL on emulated console games begins with DQN [21]. Our setting is the opposite end of that axis: one boss, one harness, 509.6 million steps across the whole project for $1,381, and a question about which of five method families does better rather than how far one of them can be pushed. The comparison we can make at this scale and they cannot easily make at theirs is a matched one.

Our imitation results are an instance of two known effects. Ross and Bagnell [26], and then DAgger [27], established that cloning's errors compound because the learner's own mistakes carry it into states the expert never visited, with regret quadratic in the horizon. That is §6.1, and 81% of our clone's episodes ending in death is what it looks like at a 154 second horizon. Weihs, Jain and colleagues [36] named the **imitation gap**: when the teacher acts on information or capability the student lacks, imitation is actively

harmful and can be beaten by pure RL. Our fifteen arms are an instance of it, with one feature their canonical case does not have. Their teacher is *privileged*, seeing what the student cannot; our human sees exactly the observation the policy sees, so the gap is capability and strategy rather than information. And the usual framing has demonstrations rescuing a weak learner, where ours are a far stronger demonstrator, 100 wins from 100, degrading a policy that is worse at the task but better at its own state distribution. ADVISOR's per-state weighting is the obvious thing we did not try.

Our shaping failure is the textbook one. Ng, Harada and Russell [22] proved that only a potential difference leaves the optimal policy unchanged. Our shaping is not potential-based, and the consequence was measured rather than predicted: a shaping cluster reached 88% of total reward magnitude, the policy optimised it correctly by driving red-eye aim to 3.36 times chance, and still died 64.5% of the time with the boss at 8 of 10 health.

Masked prediction over decision data is not ours either. MTM [37] already trains one network to reconstruct trajectories from random subsets of themselves, serving as forward model, inverse model or offline agent depending on the mask. We claim two narrower things: the masking *structure*, over an entity-by-field grid of raw engine fields with whole-actor tubes, field-group tubes and time blocks rather than random subsets; and the **free-signal floor**, a ridge fit on the cells still visible, which matters on a state vector where speed is the norm of velocity and position is the previous position plus velocity, so that a masked cell is frequently exact arithmetic on a visible one.

Losing to persistence is a known and under-reported failure, and it is being systematised as we write. A NeurIPS 2026 workshop paper, "Your world model may be worse than copying the last frame" [31], reports that of 20 recent world-model papers, 12 give a prediction-quality number and **9 of those give nothing to read it against**. The copy-last-frame floor has been standard in video prediction for years and is known to beat learned models on mostly static data. So §7 confirms rather than discovers. What we add is sharper: our model is zero-initialised so that it **begins exactly at persistence**, and 120 epochs then move it away at -0.0874 ± 0.0218 per epoch, while the action-shuffle control reads unchanged, and the failure is carried through a closed loop, on six smaller models from the same trainer, to 0 defeats in 607 real episodes. Starting at the floor and walking away from it is a stronger statement than failing to reach it.

Most of our measurement section is confirmation, and we would rather say so. Henderson et al. [18] and Agarwal et al. [1] established that few-seed point estimates mislead, that aggregates need interval estimates, and that conclusions flip when this is done properly. Our four measured noise floors, our winner's curse, and our selector-resolution arithmetic are all instances. Two things in §9 we have **not** found stated elsewhere and offer cautiously, inviting correction. First, that a metric averaged over a start distribution the experimenter both **chooses and optimises under** is not a property of the policy, which is adjacent to Agarwal's critique of aggregation across tasks but concerns initial conditions that are themselves part of the objective. Second, the end-to-end case where a sparse binary outcome concealed an 8.7-fold improvement and then motivated two measurably harmful reward changes.

What the paper offers that we have not found elsewhere is therefore not an algorithm. It is a task whose causal chain is measured rather than asserted, from 100 human victories, including the trap that the engine's own "vulnerable" flag labels 1.3% of real damage events; and a matched head-to-head of five method families on one task, one harness, one episode cap and one graded metric. Breadth at matched instruments is the contribution, not any single arm.

15 Conclusion

An agent reading a commercial game's memory learns real combat: it clears a three-enemy encounter from all ten local start states, one watched episode each and so qualitative rather than a rate, and on a dungeon boss it manufactures stuns, clears larvae and lands ten-hit kills on 8.33% of evaluation episodes across six

trained starts, reaching 28.95% from the most favourable and 0 of 76 from the least.

It does so without the reward for winning ever reaching the decisions that cause it. At the credit horizon used throughout, the kill bonus arrives at the first decision of the episode at 0.000% of face value, so what the policy knows it learned from local shaping.

One hundred human victories did not help. Cloning them reaches 74.5% next-action accuracy and 0.33 of 10 health. Of fifteen mechanisms for folding them into the policy gradient, none trends toward the policy trained without them; one arm's single-machine mean nominally exceeds it, inside a spread that one configuration run twice already covers. The mechanism that demonstrably worked cost damage at t = -4.98. Offline return-conditioned learning on the same games plus the agent's own failures does substantially better than cloning and still loses to the policy gradient by 14 standard errors. A world model trained to 120 epochs ends further from "nothing changes" than it began and ignores the action entirely, and policies dreamed inside six smaller models from the same trainer won 0 of 607 real episodes.

What remains out of reach is not the last six health points: the policy takes all ten in the episodes it wins. It is reliability. Competence arrived before consistency, and the honest summary of this report is that we know a great deal about why, mostly because we eventually started measuring the right quantity.

References

[1] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville and M. G. Bellemare. Deep Reinforcement Learning at the Edge of the Statistical Precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2108.13264.

[2] M. Andrychowicz et al. What Matters for On-Policy Deep Actor-Critic Methods? A Large-Scale Study. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2006.05990.

[3] M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA). In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 15619–15629. arXiv:2301.08243.

[4] A. Bardes et al. Revisiting Feature Prediction for Learning Visual Representations from Video (V-JEPA). *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2404.08471.

[5] M. G. Bellemare, Y. Naddaf, J. Veness and M. Bowling. The Arcade Learning Environment: An Evaluation Platform for General Agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013. arXiv:1207.4708.

[6] Y. Burda, H. Edwards, A. Storkey and O. Klimov. Exploration by Random Network Distillation (RND). In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1810.12894.

[7] L. Chen et al. Decision Transformer: Reinforcement Learning via Sequence Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)* 34, 2021. arXiv:2106.01345.

[8] K. Cho et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014. arXiv:1406.1078.

[9] J. Chung, C. Gulcehre, K. Cho and Y. Bengio. Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling. NIPS Deep Learning and Representation Learning Workshop, 2014. arXiv:1412.3555.

[10] A. Ecoffet, J. Huizinga, J. Lehman, K. O. Stanley and J. Clune. First Return, Then Explore (Go-Explore). *Nature*, 590:580–586, 2021. DOI 10.1038/s41586-020-03157-9.

[11] S. Emmons, B. Eysenbach, I. Kostrikov and S. Levine. RvS: What is Essential for Offline RL via Supervised Learning? In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2112.10751.

[12] L. Fan et al. MineDojo: Building Open-Ended Embodied Agents with Internet-Scale Knowledge. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks*, 2022. arXiv:2206.08853.

[13] V. Firoiu, W. F. Whitney and J. B. Tenenbaum. Beating the World's Best at Super Smash Bros. with Deep Reinforcement Learning. *arXiv preprint arXiv:1702.06230*, 2017.

[14] W. H. Guss et al. MineRL: A Large-Scale Dataset of Minecraft Demonstrations. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019. arXiv:1907.13440.

[15] D. Ha and J. Schmidhuber. World Models. *arXiv preprint arXiv:1803.10122*, 2018.

[16] D. Hafner, T. Lillicrap, J. Ba and M. Norouzi. Dream to Control: Learning Behaviors by Latent Imagination (Dreamer). In *International Conference on Learning Representations (ICLR)*, 2020. arXiv:1912.01603.

[17] D. Hafner, J. Pasukonis, J. Ba and T. Lillicrap. Mastering Diverse Control Tasks through World Models (DreamerV3). *Nature*, 640:647–653, 2025. arXiv:2301.04104.

[18] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup and D. Meger. Deep Reinforcement Learning that Matters. In *AAAI Conference on Artificial Intelligence (AAAI)*, 32(1):3207–3214, 2018. arXiv:1709.06560.

[19] J. Kieto. Learning in Dreams, Winning in Reality: A Continuous Dyna Loop for a Ten-Hero MOBA. *arXiv preprint arXiv:2610.08033*, 2026.

[20] D. P. Kingma and J. Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

[21] V. Mnih et al. Human-Level Control through Deep Reinforcement Learning. *Nature*, 518:529–533, 2015. DOI 10.1038/nature14236.

[22] A. Y. Ng, D. Harada and S. Russell. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.

[23] A. Nichol, V. Pfau, C. Hesse, O. Klimov and J. Schulman. Gotta Learn Fast: A New Benchmark for Generalization in RL. *arXiv preprint arXiv:1804.03720*, 2018.

[24] OpenAI: C. Berner et al. Dota 2 with Large Scale Deep Reinforcement Learning (OpenAI Five). *arXiv preprint arXiv:1912.06680*, 2019.

[25] M. Pleines, M. Pallasch, F. Zimmer and M. Preuss. Generalization, Mayhems and Limits in Recurrent Proximal Policy Optimization. *arXiv preprint arXiv:2205.11104*, 2022.

[26] S. Ross and J. A. Bagnell. Efficient Reductions for Imitation Learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 9:661–668, 2010.

[27] S. Ross, G. J. Gordon and J. A. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (DAgger). In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 15:627–635, 2011. arXiv:1011.0686.

[28] J. Schrittwieser et al. Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero). *Nature*, 588:604–609, 2020. arXiv:1911.08265.

[29] J. Schulman, P. Moritz, S. Levine, M. Jordan and P. Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation (GAE). In *International Conference on Learning Representations (ICLR)*, 2016. arXiv:1506.02438.

[30] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov. Proximal Policy Optimization Algorithms (PPO). *arXiv preprint arXiv:1707.06347*, 2017.

[31] M. Surazhsky. Your World Model May Be Worse Than Copying the Last Frame. NeurIPS 2026 Workshop on Robot Learning with World Models, 2026. https://neurips.cc/virtual/2026/166122.

[32] R. S. Sutton. Dyna, an Integrated Architecture for Learning, Planning, and Reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991. DOI 10.1145/122344.122377.

[33] R. Villegas, J. Yang, S. Hong, X. Lin and H. Lee. Decomposing Motion and Content for Natural Video Sequence Prediction. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1706.08033.

[34] R. Villegas, A. Pathak, H. Kannan, D. Erhan, Q. V. Le and H. Lee. High Fidelity Video Prediction with Large Stochastic Recurrent Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1911.01655.

[35] O. Vinyals et al. Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning (AlphaStar). *Nature*, 575:350–354, 2019. DOI 10.1038/s41586-019-1724-z.

[36] L. Weihs et al. Bridging the Imitation Gap by Adaptive Insubordination (ADVISOR). In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2007.12173.

[37] P. Wu et al. Masked Trajectory Models for Prediction, Representation, and Control (MTM). In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2305.02968.

A Artefact provenance

Every model this report draws a conclusion from, identified by content rather than by filename. The md5s and byte counts are measured, not transcribed: the full register in `artifacts/MANIFEST.json` covers 122 artefacts including datasets, logs and result rows, and records for each whether it is present locally, present in remote storage, and obtainable. The reason for matching by hash is in §13: four world-model checkpoints exist in our remote storage under the same filenames as the ones below, at roughly a third of the size, and they are an earlier run.

Table 20: The six models the report's claims rest on. Generated from the artefact register rather than typed, so a file that is replaced cannot keep an old hash in the paper.

artefact	what it is	used by	md5	bytes
`checkpoints/kept/deku-combat-gru-v2b-20p_u00120.pt`	the solved melee policy	§3	`51cd69f624b82f12`	7,419,801
`checkpoints/kept/gohma-kl-v15-r4_latest.pt`	the best boss policy, v15 rank 4	§5, §6, §9	`397e27b2beb688b9`	8,184,685
`checkpoints/kept/bc_human_100_strat.pt`	the best behaviour clone	§6.1	`6408314168deb916`	2,729,213
`checkpoints/kept/rvs_M5_s5.pt`	the best offline return-conditioned policy	§6.4	`b4aafda19665af9f`	2,732,188
`recovered/gohma-wm-r1/runs/wm_obs/wm.pt`	the observation-space world model	§7	`ba85403916e16883`	127,202,146
`runs/sjepa/p1.last.pt`	the masked-prediction encoder	§8	`fcad90898b4c9308`	9,881,105

The release copies on Hugging Face carry the same hashes. The four world-model and two token-model checkpoints not listed here are in the register with their own hashes; they are intermediate results that no claim in this report depends on.